

PHASE 5:

PROJECT DEVELOPMENT PHASE

1. INTRODUCTION TO THE DEVELOPMENT SYSTEM

The development phase of the ComicCraft application focuses on building a clean and flexible web platform. The main goal is to create a system that can take a simple text story from a user and automatically turn it into a beautiful comic book. The entire system is built using the FastAPI web framework, which makes the application very fast and responsive.

To keep the project organized and to prevent errors, the code is split into separate and independent files. Each file has a clear and specific job to do in the background. This smart structural separation ensures that when one part of the program is talking to the AI models, it does not slow down the other parts of the website, giving the user a very smooth experience.

2. CORE CODE MODULES AND TECHNICAL SPECIFICATIONS

1.main.py

This is the starting point and launcher of the entire project. Its main job is to boot up the FastAPI server using Uvicorn. It sets up the server configurations so the website can run locally on your computer at port 8000. It also configures and mounts the static folder directories so the browser can easily access and show the images and PDF files generated by the system later.

2.app/routes.py

This file acts as the main traffic controller hub for the website. When a user enters their story ideas, character names, settings, tones, and art styles on the front page and clicks submit, this script captures all that information. It connects the user interface with the backend AI models, guides the generation process step-by-step, and manages error boundaries using clean fallback functions if something goes wrong.

3.app/gemini_flash.py

This script manages the first stage of the AI pipeline. It connects directly to the Google Gemini 1.5 Flash model. It takes the user's raw story prompt and quickly breaks it down into a neat 5-panel structural blueprint outline. It ensures that the text output comes back in a clean format containing the panel index numbers, titles, and exact image description details for the next steps.

4.app/gemini_pro.py

This module manages the creative writing stage of the project using the powerful Google Gemini 1.5 Pro model. It takes the basic 5-panel layout outline generated by the previous step and expands it into a rich comic book script. It creates engaging environmental narration blocks for the background story and writes matching character dialogues and speech bubbles for each panel frame.

5.app/image_generator.py

This file is the visual art farm manager of the project. It handles the image generation workflow by taking the art description prompts and sending them to the Stable Diffusion image engine. It processes the style choices like anime, realistic, or comic book to render custom high-resolution illustrations, downloading and caching them safely as local JPEG files on the computer's hard disk.

6.app/layout_builder.py

This script acts as the structural assembler module in the system's memory. It takes the text script dialogues from the writing engine and the local image file paths from the graphic generator and stitches them together frame-by-frame. It organizes everything into a uniform chronological list layout so that the presentation layers can show the comic preview nicely on the screen.

7.app/exporters.py

This is the printing publisher engine of the application built on top of the FPDF library. It loops through the synchronized layout data to compute exact page dimensions. It automatically places the headings, inserts the local panel images, and cleanly wraps long text lines within page margins to build a print-ready, timestamped digital comic book PDF document saved on the disk.

3. DEVELOPMENT PIPELINE WORKFLOW AND DATA TRACKING

When the application is running, data travels through a continuous, well-mapped production chain. The frontend form captures the user prompt, name, and style filters, sending the dataset straight to the backend handlers. Gemini Flash instantly creates the panel blueprint matrix, Gemini Pro scripts the character text, and Stable Diffusion downloads the matching visual graphic pictures onto the disk drive. The layout engine blends these parts together into a unified dataset array list, letting the FPDF canvas manage text boundaries and print a beautiful compiled vector PDF file directly into local storage.